

《编译原理》实验报告（中间代码生成）

学号：12303070250 姓名：黄彬 时间：2026年5月8日

1 实验目的

深入理解属性文法及语法制导翻译的核心原理；掌握 Sample 语言中各种程序结构（如声明、赋值、循环、分支及函数调用）到四元式的映射方法；熟练运用拉链回填技术处理非线性控制流。

2 实验内容

针对语法、语义无误的源程序，通过对抽象语法树（AST）的递归遍历，生成满足实验要求的四元式序列。核心目标是实现一套能够处理复杂优先级、布尔短路逻辑、嵌套循环以及跨函数调用参数传递的中间代码生成引擎。

3 实验方案

3.1 方案描述（双遍 AST 遍历与拉链回填）

为了保证翻译的准确性，本实验采用了基于抽象语法树（AST）的访问者遍历方案。具体逻辑如下：

1. **表达式优先级管理**：通过下降解析器构建出严格遵循运算符结合律与优先级的 AST。特别地，针对布尔逻辑运算符（&&, ||, !）采用了“双重翻译策略”：在赋值语句中作为常规二元运算求值，在控制流语句中则触发短路跳转逻辑。

2. **控制流与回填（Backpatching）**：采用拉链回填技术。在处理 if、while、for 等结构时，先生成目标地址未知的跳转指令，并将其索引记录在 truelist 或 falselist 中，待目标标号确定后再统一进行地址更新。

3. **函数调用规范**：严格遵循 para 传参和 call 调用的四元式顺序，确保局部作用域内的变量通过四元式序列能够正确地与函数入口衔接。

3.2 方案对比分析

维度	单遍 SDT 方案	AST 遍历方案（本实验选择）
逻辑耦合度	语法分析与代码生成高度耦合，不便调试。	模块化程度高，语法树构建与代码生成完全解耦。

短路求值处理	在解析过程中处理回填链条极其复杂。	处理自然，可根据父节点类型决定子树的翻译模式。
右结合赋值	难以直接处理 $a = b = 0$ 等连续赋值。	天然支持，通过树的后序遍历轻松实现右结合逻辑。

4 实验验证与分析

在开发过程中，我们预先分析了 Sample 语言可能存在的复杂应用场景，并针对性地优化了生成逻辑。通过对五类典型代码片段的自动化验证，证明了本方案的稳健性：

场景类别	程序逻辑预判与技术对策	翻译核心逻辑（部分四元式示例）	验证结果
复杂算术	考虑到长表达式中运算符优先级和括号的深度嵌套。采用后序遍历，确保临时变量自增顺序。	('+', 'a', 'b', 't1') ('-', 't1', 't3', 't4')	符合预期
逻辑计算	分析得出赋值语句中的布尔运算不应产生跳转，而是作为普通计算。引入布尔值求值节点。	('&&', 't2', 't3', 't4') (' ', 't4', 't7', 't8')	符合预期
短路控制流	关键难点： 预判到 if/while 条件中的逻辑运算符需翻译为跳转。利用 truelist 进行拉链管理。	('j>', 'x', '0', 5) ('j', '_', '_', 9)	符合预期
嵌套循环	分析多重循环下的 break/continue 逻辑。采用循环状态栈管理回填链，支持精准跳出。	('j', '_', '_', loop_start) ('j', '_', '_', loop_end)	符合预期
函数传参	预判到参数压栈顺序对函数执行的影响。统一采用 para arg, _, _ 格式确保调用一致性。	('para', 'g', '_', '_') ('call', 'func', '_', 't1')	符合预期

5 实验结论

本次实验的核心收获在于**拉链回填技术**在非线性代码生成中的灵活应用。通过对布尔表达式在不同语境（数值语境 vs 控制流语境）下的差异化处理，我深刻认识到 AST 在处理上下文敏感翻译时的巨大优势。程序最终能够精准生成跳转标号，且四元式各分量的引号、下划线及临时变量编号均符合标准规范，为编译器后端的优化打下了坚实基础。